

# Quantara Vultra — Phased Task Breakdown

Each phase has a clear exit criteria — don't move to the next phase until the current one's criteria are met, even though the end product is full-scope (all pairs, both asset classes, RL, etc.).

## Phase 1 — Foundation & Contracts

**Goal:** Define the shared interfaces everything else builds on.

- ☐ Define the internal normalized candle schema (`schema.py`)
- ☐ Define `BaseAdapter` interface (connect, subscribe\_candles, fetch\_historical, place\_order)
- ☐ Define `BaseStrategy` interface (generate\_signal)
- ☐ Set up project structure, config management (API keys, pair lists, risk limits)
- ☐ Set up SQLite/Parquet storage schema for candles + decision logs

**Exit criteria:** All interfaces defined and documented; no exchange-specific code written yet.

## Phase 2 — First Data Pipeline (Binance, Crypto)

**Goal:** Get real, clean, multi-timeframe data flowing for one exchange.

- ☐ Build `binance_adapter.py` using `ccxt.pro`, normalize into schema
- ☐ Build `aggregator.py` — derive 15min and 4h candles from the 1min stream
- ☐ Wire ingestion → storage, confirm candles land correctly across all three timeframes
- ☐ Handle reconnection/backoff for dropped websocket connections

**Exit criteria:** Continuous, reliable 1min/15min/4h candle data for at least a handful of Binance pairs, stored and inspectable.

## Phase 3 — Feature Engineering & Regime Detection

**Goal:** Turn raw candles into a working regime classifier.

- ☐ Build `feature_engineering.py` (realized vol, Hurst exponent, ATR, autocorrelation, volume z-score)
- ☐ Pull historical 4h data, compute features, build training dataset
- ☐ Train Gaussian HMM (`train_hmm.py`) with 5+ components
- ☐ Manually inspect and label the resulting regimes based on their feature signatures
- ☐ Build `regime_model.py` for live inference (outputs probability distribution, not hard labels)

**Exit criteria:** Given live 4h data for a pair, the system outputs a sensible, human-checkable regime probability distribution.

## Phase 4 — First Strategy End-to-End (Single Pair, Paper Trading)

**Goal:** Prove the full decision chain works before scaling anything.

- ☐ Build one strategy (`momentum.py`) using 15min direction + 1min entry timing
- ☐ Build `allocator.py` combining regime probabilities with the one strategy's signal
- ☐ Build `risk_gate.py` — confidence threshold, position sizing, drawdown limits
- ☐ Test the risk gate against synthetic bad scenarios (low confidence, high drawdown) — confirm it correctly blocks trades
- ☐ Build `paper_executor.py`, wire the full chain together for ONE pair
- ☐ Build the structured decision/trade logger

**Exit criteria:** One pair, fully paper-trading, with every decision (regime → signal → allocation → risk check → execution) logged and inspectable end-to-end.

## Phase 5 — Scale Out: All Strategies, All Crypto Pairs

**Goal:** Go from "it works on one pair" to full crypto scope.

- ☐ Build remaining strategies (`mean_reversion.py`, `breakout.py`, `defensive.py`)
- ☐ Build the asyncio orchestrator to run the pipeline across all Binance pairs in parallel
- ☐ Add `coinbase_adapter.py` and any other exchange adapters, same interface pattern
- ☐ Confirm rate-limit handling, backoff, and logging hold up under full pair-count load

**Exit criteria:** Full crypto scope (all pairs, all exchanges) running in paper trading simultaneously, with per-pair logs distinguishable from each other.

## Phase 6 — Add Stocks (Alpaca)

**Goal:** Extend the proven pipeline to a second asset class.

- ☐ Build `alpaca_adapter.py` using the same `BaseAdapter` interface
- ☐ Add market-hours awareness to the orchestrator (stocks aren't 24/7 like crypto)
- ☐ Confirm stock pairs flow through the exact same regime/strategy/allocator/risk pipeline with no logic changes needed

**Exit criteria:** Stocks and crypto both running in parallel through the same pipeline, in paper trading.

## Phase 7 — Backtesting Bias Gate

**Goal:** Build the validation layer before letting anything self-modify.

- ☐ Build `bias_gate.py` implementing all 6 checks (survivorship, look-ahead, overfitting, data-snooping, transaction-cost, selection bias)
- ☐ Test it against a deliberately overfit/biased strategy — confirm it correctly rejects it
- ☐ Wire it as a mandatory gate for any strategy or parameter change before it can go live

**Exit criteria:** A known-bad strategy is reliably caught and rejected by the gate.

## Phase 8 — Reinforcement Learning Layer

**Goal:** Let the system start improving itself, safely.

- ☐ Define state space (regime probs + recent performance per pair/strategy)
- ☐ Define action space (parameter adjustments, allocator weight adjustments)
- ☐ Build the asymmetric reward function (losses weighted more heavily than gains)
- ☐ Start with a simple policy-gradient or Q-learning setup — not deep RL
- ☐ Route every RL-proposed update through the Bias Gate, then the Risk Gate, before it's applied

**Exit criteria:** RL proposes at least one parameter change that passes both gates and measurably improves paper-trading performance over a testing window.

## Phase 9 — Live Capital (only after sustained paper-trading validation)

**Goal:** Move from simulated to real execution, carefully.

- ☐ Build `live_executor.py` (crypto via ccxt, stocks via Alpaca)
- ☐ Start with minimal capital and a small subset of pairs, not full scope immediately
- ☐ Compare live fills/performance against paper-trading predictions systematically
- ☐ Only scale up capital and pair coverage once live behavior matches paper behavior within acceptable tolerance

**Exit criteria:** Live performance is being actively tracked against paper predictions, with no unexplained divergence.

## How to use this

Work top to bottom. Phases 5, 6, 7, and 8 can be reordered somewhat based on what you want to prioritize (e.g. you could add stocks before scaling out all crypto pairs), but Phases 1–4 must come first in order, and Phase 9 (live capital) must always come last.